



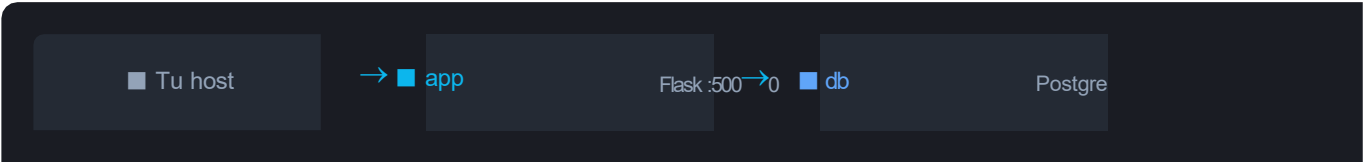
Práctica Guiada de Docker

App Flask + PostgreSQL · Red personalizada · Volumen persistente

Esta práctica te guía paso a paso para levantar manualmente, sin docker-compose, una aplicación Python (Flask) conectada a una base de datos PostgreSQL, usando una red personalizada y un volumen persistente.

■ Tecnologías	Docker Engine · Python 3.12 · Flask 3 · PostgreSQL 16
■ Red	<code>docker network create --driver bridge lab-net</code>
■ Volumen	<code>docker volume create pgdata</code>
■ App	Imagen personalizada construida con docker build
■ Acceso	Desde el host en <code>http://localhost:5000</code>
■ Duración	~30–45 minutos

ARQUITECTURA



■ ■ red: lab-net (bridge) · volumen: pgdata ■ ■

1

Crear la red y el volumen

Infraestructura base

Antes de correr cualquier contenedor creamos la red personalizada y el volumen de datos. La red permite que los contenedores se comuniquen por nombre (db en vez de una IP dinámica). El volumen garantiza que los datos de PostgreSQL sobrevivan si el contenedor se reinicia o borra.

Crear la red

```
# Crear red tipo bridge llamada 'lab-net'
docker network create --driver bridge lab-net
a3f8b1c2d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9
```

Crear el volumen

```
# Crear volumen para persistir datos de PostgreSQL
docker volume create pgdata
pgdata
```

- La red lab-net permite resolución de nombres: el contenedor 'app' podrá conectarse a 'db' simplemente usando ese nombre como hostname, sin IPs.

VERIFICACIÓN

verificar creación

```
docker network ls
NETWORK ID          NAME        DRIVER  SCOPE
xxxxxxxxxxxxx lab-net    bridge  local  ✓
docker volume ls
DRIVER  VOLUME NAME
local   pgdata  ✓
```

```
PS C:\Users\josem> docker network create --driver bridge lab-net
9296f0e60b3e18d88876750935981299474aaaf38dc7747c10087a1e8e8f64ae
PS C:\Users\josem> docker network ls
NETWORK ID          NAME        DRIVER  SCOPE
775fbff3b143 app_red    bridge  local
52a2908eb214 bridge    bridge  local
e2f7fd92aaf3 host      host    local
9296f0e60b3e lab-net    bridge  local
a8296eacd3ed mi_red    bridge  local
1ea06241dbb9 none      null    local
PS C:\Users\josem> docker volume create pgdata
pgdata
PS C:\Users\josem> docker volume ls
DRIVER  VOLUME NAME
local   1d5cef87275614f0c75dd501a645b2709af82879815ac058958f91ffaa703e6f
local   2ac57fdd2d64c28b08e445036b9c2a6a273a6ff0a4dedbc90b719ae4e32409ca
local   6b4b675330b9b67db7829209e8fda3de33c4e188c26f0b63e70f52732e53f718
local   9891e58777489cf8f3272a15cf84d2b0f4ba8523d8af2737290d815cc1aa3784
local   c03c99d099a77cf16ff01d98c97800d174d179bc257dcefb50ef378bf5227b9
local   ce3a561a8481d1ccbb9371600ac1de65b2197f7be77567765298e4cdaab211af2
local   d4e6cd960002b19bd808ddbe8176131f05a54d6a63251c8ec7d3f6e42e3aac67
local   e3cb1e1aadf8b6a81ffd2873e41f16cc610c8071766e2b52c0b8ad201cc86f35
local   e0853b6dc2a47481a1fee45bf4969288871ed13965bd26634ad6cb32ccae3dfd
local   fb19c1fac8608784c7a67260e98e2ba28bcf4346ba155d314d165b7c09679cfd
local   mysql_datos
local   pgdata
PS C:\Users\josem>
```

2 Levantar PostgreSQL

Contenedor de base de datos

Corremos PostgreSQL 16 Alpine (imagen liviana) conectado a nuestra red y con el volumen montado. El flag `--name db` es clave: así Flask lo encontrará por ese nombre en la red.

```
docker run -d \
  --name db \
  --network lab-net \
  --mount source=pgdata,target=/var/lib/postgresql/data \
  -e POSTGRES_USER=admin \
  -e POSTGRES_PASSWORD=secret123 \
  -e POSTGRES_DB=appdb \
  postgres:16-alpine
b7e3f2a1c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9
```

<code>-d</code>	Corre en background (detached mode)
<code>--name db</code>	Nombre del contenedor — Flask lo usará como hostname
<code>--network</code>	Conecta el contenedor a la red lab-net
<code>--mount</code>	Monta el volumen pgdata en la ruta de datos de Postgres
<code>-e POSTGRES_*</code>	Variables de entorno para configurar usuario y BD

VERIFICACIÓN

```
docker ps
CONTAINER ID   IMAGE                STATUS              NAMES
xxxxxxxxxxxx   postgres:16-alpine  Up 10 seconds      db ✓
# Ver logs (esperar ~5 segundos)
docker logs db
...database system is ready to accept connections ✓
# Conectarse directamente (opcional)
docker exec -it db psql -U admin -d appdb
appdb=# \q
```

```
PS C:\Users\josem> docker run -d --name db --network lab-net --mount source=pgdata,target=/var/lib/postgresql/data -e POSTGRES_USER=admin -e POSTGRES_PASSWORD=secret123 -e POSTGRES_DB=appdb postgres:16-alpine
Unable to find image 'postgres:16-alpine' locally
16-alpine: Pulling from library/postgres
3b7e6bf074f6: Pull complete
282a6867e326: Pull complete
20f9cf2e9893: Pull complete
420ca0de84ca: Pull complete
0d3e610f9e0f: Pull complete
72393ada9150: Pull complete
c740b6b7dd0b: Pull complete
d9681cd68a94: Pull complete
5ef55a6c860c: Pull complete
abdc7c6150b5: Pull complete
9827fda4490e: Download complete
fb2a3b9ecbd5: Download complete
Digest: sha256:4e6e670bb069649261c9c18031f0aded7bb249a5b6664ddec29c013a89310d50
Status: Downloaded newer image for postgres:16-alpine
a54f3ff629ac2795747cd15168618b8d30c48f97476255ed666c925d497d1534
PS C:\Users\josem> docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS        NAMES
a54f3ff629ac   postgres:16-alpine   "docker-entrypoint.s..." 37 seconds ago Up 34 seconds 5432/tcp     db
0c0b1d45422d   postgres             "docker-entrypoint.s..." 6 hours ago   Up 6 hours   5432/tcp     mi_postgres
```

```
PS C:\Users\josem> docker logs db
The files belonging to this database system will be owned by user "postgres".
This user must also own the server process.

The database cluster will be initialized with locale "en_US.utf8".
The default database encoding has accordingly been set to "UTF8".
The default text search configuration will be set to "english".

Data page checksums are disabled.

fixing permissions on existing directory /var/lib/postgresql/data ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default max_connections ... 100
selecting default shared_buffers ... 128MB
selecting default time zone ... UTC
creating configuration files ... ok
running bootstrap script ... ok
sh: locale: not found
2026-05-04 23:14:43.794 UTC [35] WARNING: no usable system locales were found
performing post-bootstrap initialization ... ok
initdb: warning: enabling "trust" authentication for local connections
syncing data to disk ... ok

Success. You can now start the database server using:

    pg_ctl -D /var/lib/postgresql/data -l logfile start

initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the next time you run initdb.
waiting for server to start...2026-05-04 23:14:45.767 UTC [41] LOG: starting PostgreSQL 16.13 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
2026-05-04 23:14:45.775 UTC [41] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2026-05-04 23:14:45.822 UTC [44] LOG: database system was shut down at 2026-05-04 23:14:45 UTC
2026-05-04 23:14:45.843 UTC [41] LOG: database system is ready to accept connections
done
server started
CREATE DATABASE

/usr/local/bin/docker-entrypoint.sh: ignoring /docker-entrypoint-initdb.d/*

waiting for server to shut down...2026-05-04 23:14:46.151 UTC [41] LOG: received fast shutdown request
2026-05-04 23:14:46.156 UTC [41] LOG: aborting any active transactions
2026-05-04 23:14:46.162 UTC [41] LOG: background worker "logical replication launcher" (PID 47) exited with exit code 1
2026-05-04 23:14:46.163 UTC [42] LOG: shutting down
2026-05-04 23:14:46.166 UTC [42] LOG: checkpoint starting: shutdown immediate
2026-05-04 23:14:46.318 UTC [42] LOG: checkpoint complete: wrote 926 buffers (5.7%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.048 s, sync=0.099 s, total=0.155 s; sync files=301, longest=0.004 s, average=0.001 s; distance=4272 kb, estimate=4272 kb; lsn=0/191E928, redo lsn=0/191E928
2026-05-04 23:14:46.329 UTC [41] LOG: database system is shut down
done
server stopped

PostgreSQL init process complete; ready for start up.

2026-05-04 23:14:46.392 UTC [1] LOG: starting PostgreSQL 16.13 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
2026-05-04 23:14:46.394 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
2026-05-04 23:14:46.395 UTC [1] LOG: listening on IPv6 address ":::", port 5432
2026-05-04 23:14:46.401 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2026-05-04 23:14:46.419 UTC [9] LOG: database system was shut down at 2026-05-04 23:14:46 UTC
2026-05-04 23:14:46.431 UTC [1] LOG: database system is ready to accept connections
PS C:\Users\josem> |
```

```
PS C:\Users\josem> docker exec -it db psql -U admin -d appdb
psql (16.13)
Type "help" for help.

appdb=# \q
```

What's next:

Try Docker Debug for seamless, persistent debugging tools in any container or image → [docker debug db](#)
 Learn more at <https://docs.docker.com/go/debug-cli/>

```
PS C:\Users\josem>
```

3

Crear la aplicación Flask

Código y Dockerfile

Creamos tres archivos en una carpeta mi-app/: la aplicación Flask, los requirements de Python y el Dockerfile para construir la imagen.

● ● ● Crear carpeta y archivos

```
mkdir mi-app && cd mi-app
touch app.py requirements.txt Dockerfile
# Abre cada archivo con tu editor y pega el contenido de abajo ✓
```

```
PS C:\Users\josem\mi-app2> New-item app.py, requirements.txt, Dockerfile
```

Directory: C:\Users\josem\mi-app2

Mode	LastWriteTime	Length	Name
-a	5/4/2026 7:24 PM	0	app.py
-a	5/4/2026 7:24 PM	0	requirements.txt
-a	5/4/2026 7:24 PM	0	Dockerfile

■ app.py

```
from flask import Flask, jsonify
import psycopg2

app = Flask(__name__)

def get_conn():
    return psycopg2.connect(
        host="db",          # nombre del contenedor en la red
        dbname="appdb", user="admin", password="secret123"
    )

@app.route("/")
def home():
    conn = get_conn()
    cur = conn.cursor()
    cur.execute("SELECT version();")
    ver = cur.fetchone()[0]
    cur.close(); conn.close()
    return jsonify({"status": "ok", "pg_version": ver})

@app.route("/visitas")
def visitas():
    conn = get_conn()
    cur = conn.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS visitas
            (id SERIAL, ts TIMESTAMPTZ DEFAULT now());
        INSERT INTO visitas DEFAULT VALUES;
        SELECT COUNT(*) FROM visitas;""")
    conn.commit()
    total = cur.fetchone()[0]
    cur.close(); conn.close()
    return jsonify({"total_visitas": total})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

■ requirements.txt

```
requirements.txt

flask==3.0.3
psycpg2-binary==2.9.9
```

■ Dockerfile

```
Dockerfile

FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```


4

Construir la imagen

docker build — crear imagen personalizada

Construimos la imagen de nuestra app Flask desde el Dockerfile. Este proceso descarga la imagen base de Python, instala Flask y psycopg2, y empaqueta el código en una imagen reutilizable llamada mi-app:1.0.

```

docker build

# Desde dentro de la carpeta mi-app/
docker build -t mi-app:1.0 .

Step 1/6 : FROM python:3.12-slim
---> abc123ef...
Step 2/6 : WORKDIR /app
Step 3/6 : COPY requirements.txt .
Step 4/6 : RUN pip install --no-cache-dir -r requirements.txt
    Installing collected packages: flask, psycopg2-binary
Step 5/6 : COPY . .
Step 6/6 : CMD [python, app.py]
Successfully built def456ab...
Successfully tagged mi-app:1.0 ✓

```

-t mi-app:1.0

Nombre y tag de la imagen resultante

.

Usa el directorio actual como contexto de build

```

PS C:\Users\josem\mi-app> docker build -t mi-app:1.0 .
[+] Building 23.4s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 217B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.12-slim@sha256:46cb7cc2877e60fbd5e21a9ae6115c30ace7a077b9f8772da879e4590c18c2e3
=> => resolve docker.io/library/python:3.12-slim@sha256:46cb7cc2877e60fbd5e21a9ae6115c30ace7a077b9f8772da879e4590c18c2e3
=> => sha256:4d873bcef452ab1c054b9a0f25ace04a46c558a7b925aed2fcb5bc21533e0f65 12.10MB / 12.10MB
=> => sha256:3fcd9ab5045181fbd49d1d30a2dbad144172d4c3c141f70117ddb0b918d8357 248B / 248B
=> => sha256:cb8ca9140ac107575972113eb5fd2e6c3cdddc592371a5ac31c4623dcb13d5d 1.29MB / 1.29MB
=> => extracting sha256:cb8ca9140ac107575972113eb5fd2e6c3cdddc592371a5ac31c4623dcb13d5d
=> => extracting sha256:4d873bcef452ab1c054b9a0f25ace04a46c558a7b925aed2fcb5bc21533e0f65
=> => extracting sha256:3fcd9ab5045181fbd49d1d30a2dbad144172d4c3c141f70117ddb0b918d8357
=> [internal] load build context
=> => transferring context: 278B
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY . .
=> => exporting to image
=> => exporting layers
=> => exporting manifest sha256:fdb0e7e217d34971f9b544da5a21956f8eb92346940fb7116998fd01413cd46e
=> => exporting config sha256:7ab9466557605d7b2ef7d904a50e2a03bd5397d8cce132e224290cf228cf4aed
=> => exporting attestation manifest sha256:3b3b100508d419ba485c24262e56cde2089bd8b9f37ac11c099dfecdbb7e4cfff
=> => exporting manifest list sha256:573d20ca3cc5055e3d24e5dabc7af70a44232e60b5a19924f87564bab4df8f41
=> => naming to docker.io/library/mi-app:1.0
=> => unpacking to docker.io/library/mi-app:1.0
PS C:\Users\josem\mi-app>

```

VERIFICACIÓN

```

docker images

REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
mi-app        1.0       def456ab...    1 minute ago  ~180MB ✓

```

```

PS C:\Users\josem> docker images

IMAGE                ID                DISK USAGE    CONTENT SIZE    EXTRA
app:latest           3388af227ebe     176MB         43.9MB
hello-world:latest   f9078146db2e     25.9kB        9.49kB
mi-app:1.0           573420ca3cc5     208MB         51.3MB
mi-node-app:latest   00333c1bf5d5     1.57GB        397MB
mysql:8.0            d0304ed9fdb6     1.1GB         249MB
mysql:latest         c9e48b0e008f     1.3GB         290MB
nginx:latest         6e23479198b9     240MB         65.8MB
node-app:latest      bfe97f76d5ed     1.59GB        398MB
node:latest          c69f4e0640e5     1.64GB        424MB
postgres:16-alpine   4e6e670bb069     395MB         111MB
postgres:latest      78481659c47e     649MB         168MB
PS C:\Users\josem>

```

5

Correr la app y probarla

Conectar todo y ejecutar

Lanzamos el contenedor de la app conectándolo a la misma red donde está el contenedor db. El flag -p expone el puerto 5000 del contenedor al puerto 5000 de tu máquina.

```

docker run -d \
  --name app \
  --network lab-net \
  -p 5000:5000 \
  mi-app:1.0
9a8b7c6d5e4f3a2b1c0d9e8f7a6b5c4d3e2f1a0b

```

```

probar desde el host

# Probar conexión a la BD
curl http://localhost:5000/
{"pg_version": "PostgreSQL 16.x...", "status": "ok"}

# Contar visitas - crea tabla y registra cada hit
curl http://localhost:5000/visitas
{"total_visitas": 1}
curl http://localhost:5000/visitas
{"total_visitas": 2}

# También funciona en el navegador:
http://localhost:5000/visitas

```

- ✓ La app Flask se conecta a PostgreSQL usando host='db' porque ambos contenedores comparten la red lab-net y Docker resuelve el nombre automáticamente. Desde tu host accedes por localhost:5000 gracias al flag -p 5000:5000.

```

PS C:\Users\josem> docker run -d --name app --network lab-net -p 5000:5000 mi-app:1.0
56c234eee0ba4ed0ac7c0f90d93b256debd0a2d3321c76b22f7c0bb3343bc56e
PS C:\Users\josem> curl http://localhost:5000/

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): Y

StatusCode      : 200
StatusDescription : OK
Content          : {"pg_version":"PostgreSQL 16.13 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0,
64-bit","status":"ok"}
RawContent       : HTTP/1.1 200 OK
Connection: close
Content-Length: 120
Content-Type: application/json
Date: Mon, 04 May 2026 23:46:47 GMT
Server: Werkzeug/3.1.8 Python/3.12.13

Forms            : {}
Headers          : {[Connection, close], [Content-Length, 120], [Content-Type, application/json], [Date, Mon, 04 May 2026 23:46:47 GMT] ...}
Images           : {}
InputFields      : {}
Links            : {}
ParsedHtml       : mshtml.HTMLDocumentClass
RawContentLength : 120

PS C:\Users\josem>

```



```
PS C:\Users\josem> curl http://localhost:5000/visitas

Security Warning: Script Execution Risk
Invoke-WebRequest: parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): A

StatusCode      : 200
StatusDescription : OK
Content         : {"total_visitas":1}

RawContent      : HTTP/1.1 200 OK
                  Connection: close
                  Content-Length: 20
                  Content-Type: application/json
                  Date: Mon, 04 May 2026 23:48:10 GMT
                  Server: Werkzeug/3.1.8 Python/3.12.13
                  {"total_visitas":1}

Forms           : {}
Headers         : {[Connection, close], [Content-Length, 20], [Content-Type, application/json], [Date, Mon, 04 May 2026 23:48:10 GMT] ... }
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshhtml.HTMLDocumentClass
RawContentLength : 20

PS C:\Users\josem> |
```

```
PS C:\Users\josem> curl http://localhost:5000/visitas

StatusCode      : 200
StatusDescription : OK
Content         : {"total_visitas":2}

RawContent      : HTTP/1.1 200 OK
                  Connection: close
                  Content-Length: 20
                  Content-Type: application/json
                  Date: Mon, 04 May 2026 23:48:53 GMT
                  Server: Werkzeug/3.1.8 Python/3.12.13
                  {"total_visitas":2}

Forms           : {}
Headers         : {[Connection, close], [Content-Length, 20], [Content-Type, application/json], [Date, Mon, 04 May 2026 23:48:53 GMT] ... }
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshhtml.HTMLDocumentClass
RawContentLength : 20

PS C:\Users\josem>
```

VERIFICACIÓN

```
verificar ambos contenedores

docker ps
CONTAINER ID   IMAGE          STATUS    NAMES
xxx            mi-app:1.0    Up        app ✓
yyy            postgres:16-alpine Up        db ✓
# Ver la red: ambos contenedores deben aparecer
docker network inspect lab-net
"Containers": { "app": {...}, "db": {...} } ✓
```

```
PS C:\Users\josem> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
56c234eee0ba  mi-app:1.0    "python app.py"         6 minutes ago Up 5 minutes  0.0.0.0:5000→5000/tcp, [::]:5000→5000/tcp  app
a54f3ff629ac  postgres:16-alpine "docker-entrypoint.s..." 35 minutes ago Up 35 minutes  5432/tcp  db
0c0b1d45422d  postgres      "docker-entrypoint.s..." 7 hours ago   Up 7 hours    5432/tcp  mi_postgres

PS C:\Users\josem>
```

```
{
  "com.docker.network.enable_ipv6": "false"
},
"Labels": {},
"Containers": {
  "56c234eee0ba": {
    "Name": "app",
    "EndpointID": "c069d6f6c1bade7e43efc16d1b33708a409fc545d217ce9ee12a458b97f50e61",
    "MacAddress": "1a:7e:10:e7:8b:38",
    "IPv4Address": "172.20.0.3/16",
    "IPv6Address": ""
  },
  "a54f3ff629ac": {
    "Name": "db",
    "EndpointID": "3f130729db8af91a11c1611e3e87e2574107fbbd3cd018d0ecc3daae040881c1",
    "MacAddress": "da:1d:79:d5:83:44",
    "IPv4Address": "172.20.0.2/16",
    "IPv6Address": ""
  }
},
"Status": {
```

6

Probar persistencia del volumen

El volumen sobrevive al contenedor

Esta es la parte más importante: vamos a destruir el contenedor de PostgreSQL, recrearlo con el mismo volumen pgdata y verificar que los datos siguen ahí. Esto demuestra por qué los volúmenes son esenciales en producción.

```
probar persistencia paso a paso

# 1. Acumula varias visitas
curl http://localhost:5000/visitas # repite 4-5 veces
{"total_visitas": 5}

# 2. Destruye el contenedor de la BD (¡pero NO el volumen!)
docker stop db && docker rm db
db

# 3. Recrea el contenedor con el MISMO volumen pgdata
docker run -d \
  --name db --network lab-net \
  --mount source=pgdata,target=/var/lib/postgresql/data \
  -e POSTGRES_USER=admin -e POSTGRES_PASSWORD=secret123 \
  -e POSTGRES_DB=appdb postgres:16-alpine

# 4. Espera 5 segundos y consulta - los datos siguen ahí
curl http://localhost:5000/visitas
{"total_visitas": 5} ← datos sobrevivieron al restart ✓
```

- Sin volumen, al borrar el contenedor se perderían todos los datos. Con `--mount source=pgdata` los archivos de Postgres viven en el host, independientes del ciclo de vida del contenedor.

```
PS C:\Users\josem> curl http://localhost:5000/visitas

StatusCode      : 200
StatusDescription : OK
Content         : {"total_visitas":5}

RawContent      : HTTP/1.1 200 OK
                  Connection: close
                  Content-Length: 20
                  Content-Type: application/json
                  Date: Mon, 04 May 2026 23:54:37 GMT
                  Server: Werkzeug/3.1.8 Python/3.12.13

                  {"total_visitas":5}

Forms           : {}
Headers         : {[Connection, close], [Content-Length, 20], [Content-Type
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 20

PS C:\Users\josem>
```

```

PS C:\Users\josem> docker stop db
db
PS C:\Users\josem> docker rm db
db
PS C:\Users\josem> docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED         STATUS         PORTS                               NAMES
56c234eee0ba   mi-app:1.0 "python app.py"         12 minutes ago Up 12 minutes  0.0.0.0:5000→5000/tcp, [::]:5000→5000/tcp  app
0c0b1d45422d   postgres  "docker-entrypoint.s..." 7 hours ago    Up 7 hours     5432/tcp                             mi_postgres
PS C:\Users\josem> docker run -d --name db --network lab-net --mount source=pgdata,target=/var/lib/postgresql/data -e POSTGRES_USER=admin -e POSTGRES_PASSWORD=secret123 -e POSTGRES_DB=appdb postgres:16-alpine bdc73e98100f80ef5273c49445a5e0d0c44de81f69f60c871c094cf990d89c8e
PS C:\Users\josem> curl http://localhost:5000/visitas

StatusCode      : 200
StatusDescription : OK
Content         : {"total_visitas":6}

RawContent      : HTTP/1.1 200 OK
                  Connection: close
                  Content-Length: 20
                  Content-Type: application/json
                  Date: Mon, 04 May 2026 23:57:57 GMT
                  Server: Werkzeug/3.1.8 Python/3.12.13
                  {"total_visitas":6}

Forms           : {}
Headers         : {[Connection, close], [Content-Length, 20], [Content-Type, application/json], [Date, Mon, 04 May 2026 23:57:57 GMT] ... }
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 20

PS C:\Users\josem>

```

7

Limpiar el laboratorio

Borrar todo lo creado

Al terminar la práctica, limpiamos todos los recursos creados: contenedores, imagen, red y volumen.

```

limpiar todo

# Detener y borrar contenedores
docker stop app db
docker rm app db

# Borrar imagen construida
docker rmi mi-app:1.0

# Borrar red y volumen
docker network rm lab-net
docker volume rm pgdata

Laboratorio limpio ✓

```


VERIFICACIÓN FINAL

```

verificar limpieza completa

docker ps -a
CONTAINER ID   IMAGE     ...   (vacío) ✓
docker volume ls
DRIVER         VOLUME NAME   (vacío) ✓
docker network ls
bridge        host         none        (solo las default) ✓

```

✓ ¡Práctica completada! Dominaste: `docker network create` · `docker volume create` · `docker build -t` · `docker run --network` · `docker run --mount` · `-p host:container`

```

PS C:\Users\josem> docker stop app db
app
db
PS C:\Users\josem> docker rm app db
app
db
PS C:\Users\josem> docker rmi mi-app:1.0
Untagged: mi-app:1.0
Deleted: sha256:573d20ca3cc5055e3d24e5dabc7af70a44232e60b5a19924f87564bab4df8f41
PS C:\Users\josem> docker network rm lab-net
lab-net
PS C:\Users\josem> docker volume rm pgdata
pgdata
PS C:\Users\josem> docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS    NAMES
0c0b1d45422d   postgres  "docker-entrypoint.s..."  7 hours ago  Up 7 hours  5432/tcp  mi_postgres
PS C:\Users\josem> docker volume ls
DRIVER         VOLUME NAME
local         1d5cef87275614f0c75dd501a645b2709af82879815ac058958f91ffaa703e6f
local         2ac57fdd2d64c28b08e445036b9c2a6a273a6ff0a4dedbc90b719ae4e32409ca
local         6b4b675330b9b67db7829209e8fda3de33c4e188c26f0b63e70f52732e53f718
local         9891e58777489cf8f3272a15cf84d2b0f4ba8523d8af2737290d815cc1aa3784
local         c03c99d099a77cf16ff01d98c97800d174d179bc257dcefb50ef378bf5227b9
local         ce3a561a8481d1ccbb9371600ac1de65b2197f7be77567765298e4cdaab211af2
local         d4e6cd960002b19bd808ddbe8176131f05a54d6a63251c8ec7d3f6e42e3aac67
local         e3cb1e1aadf8b6a81fffd2873e41f16cc610c8071766e2b52c0b8ad201cc86f35
local         e0853b6dc2a47481a1fee45bf4969288871ed13965bd26634ad6cb32ccae3dfd
local         fb19c1fac8608784c7a67260e98e2ba28bcf4346ba155d314d165b7c09679cfd
local         mysql_datos
PS C:\Users\josem> docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
775fbff3b143   app_red   bridge    local
52a2908eb214   bridge    bridge    local
e2f7fd92aaf3   host      host      local
a8296eacd3ed   mi_red    bridge    local
1ea06241dbb9   none      null      local
PS C:\Users\josem>

```

RESUMEN DE COMANDOS

Comando	Descripción
<code>docker network create --driver bridge NAME</code>	Crear red personalizada
<code>docker volume create NAME</code>	Crear volumen persistente
<code>docker build -t IMAGE:TAG .</code>	Construir imagen desde Dockerfile
<code>docker run --network NAME ...</code>	Conectar contenedor a una red
<code>docker run --mount source=VOL,target=PATH</code>	Montar volumen en contenedor
<code>docker run -p HOST:CONTAINER ...</code>	Exponer puerto al host
<code>docker network inspect NAME</code>	Ver contenedores en la red
<code>docker logs CONTAINER</code>	Ver logs del contenedor


```
docker exec -it CONTAINER bash
```

Terminal dentro del contenedor